



Procesos ETL, Automatización y Análisis Exploratorio de Datos

En esta clase exploraremos el flujo completo de trabajo con datos en Python, desde la extracción hasta la generación de reportes automatizados. Conectaremos el backend del análisis (ETL) con el frontend (visualización y reporte), estableciendo las bases para proyectos profesionales de ciencia de datos.

Objetivos de aprendizaje



Comprender el flujo ETL

Dominar cada etapa del proceso Extract → Transform → Load y su aplicación práctica



Identificar el rol del EDA

Entender dónde y cuándo aplicar el análisis exploratorio dentro del flujo de datos



Dominar herramientas Python

Conocer las bibliotecas esenciales para extracción, transformación y exportación de datos



Explorar técnicas avanzadas

Introducción práctica a web scraping y conexión con bases de datos SQL

¿Qué es ETL y cómo se relaciona con el análisis de datos?

ETL es el acrónimo de **Extract, Transform, Load**, un proceso fundamental que constituye la columna vertebral de cualquier proyecto de datos. Este flujo sistemático garantiza que los datos sean obtenidos, procesados y almacenados de manera consistente y confiable.



Extract (Extracción)

Obtención de datos desde múltiples fuentes: archivos locales, bases de datos, APIs o sitios web. Es el punto de partida de todo análisis.



Transform (Transformación)

Limpieza, normalización y enriquecimiento de los datos. Aquí se aplican reglas de negocio y se garantiza la calidad de la información.



Load (Carga)

Almacenamiento o exportación de los datos procesados en un formato útil para su consumo posterior.

♦ Dos ramificaciones del flujo ETL

Proyectos de análisis

El flujo continúa hacia un **EDA y visualización**. El objetivo es explorar, comprender y comunicar insights a través de gráficos y estadísticas descriptivas.

- Visualizaciones interactivas
- Reportes analíticos
- Descubrimiento de patrones

Proyectos de automatización

El flujo termina tras la **transformación y carga**. No hay análisis visual, solo procesos repetibles y confiables que generan salidas estandarizadas.

- Reportes automatizados
- Actualizaciones programadas
- Pipelines de producción

Etapa 1: Extracción (Extract)

La extracción es el primer paso crítico en cualquier proyecto de datos. Consiste en obtener información desde diversas fuentes y llevarla a un entorno donde podamos trabajar con ella. La versatilidad de Python nos permite conectarnos a prácticamente cualquier origen de datos.

Fuentes comunes de datos

Archivos locales

CSV, Excel, JSON, Parquet

Perfectos para datasets pequeños y medianos que no requieren infraestructura compleja

Bases de datos SQL

MySQL, PostgreSQL, SQLite

Ideales para datos estructurados en entornos empresariales

APIs REST

Servicios web y endpoints

Acceso programático a datos dinámicos y actualizados

Web scraping

Extracción desde HTML

Útil cuando no existe una API disponible

Ejemplos prácticos con Pandas

Extracción desde CSV

```
import pandas as pd

# Leer archivo CSV local
df = pd.read_csv("ventas.csv")

# Ver primeras filas
print(df.head())
```

La función `read_csv()` es una de las más utilizadas en ciencia de datos por su simplicidad y eficiencia.

Extracción desde SQL

```
import sqlalchemy as sa

# Crear conexión
engine = sa.create_engine(
    "sqlite:///tienda.db"
)

# Ejecutar consulta
df = pd.read_sql(
    "SELECT * FROM productos",
    engine
)
```

SQLAlchemy facilita la conexión con múltiples tipos de bases de datos usando una sintaxis unificada.

Web Scraping: Extracción de datos desde la web

El web scraping es una técnica poderosa para obtener datos de sitios web cuando no existe una API disponible. Nos permite automatizar la recopilación de información pública, convirtiendo páginas HTML en datasets estructurados listos para el análisis.

Bibliotecas populares en Python



requests

Biblioteca para realizar peticiones HTTP y descargar el contenido HTML de páginas web. Simple, elegante y ampliamente adoptada en la comunidad Python.



BeautifulSoup

Parser HTML que permite navegar y extraer información del árbol DOM. Ideal para analizar texto, tablas y elementos específicos de páginas web.



Scrapy

Framework completo para proyectos de scraping a gran escala. Incluye gestión de peticiones, parseo y almacenamiento de datos.

Ejemplo básico de scraping

```
import requests
from bs4 import BeautifulSoup

# Descargar el HTML de la página
url = "https://quotes.toscrape.com/"
response = requests.get(url)
html = response.text

# Analizar el contenido con BeautifulSoup
soup = BeautifulSoup(html, "html.parser")

# Extraer todas las citas
quotes = [q.text for q in soup.select(".text")]

# Mostrar las primeras citas
for quote in quotes[:3]:
    print(quote)
```

Consideraciones éticas y legales

Siempre respeta las reglas del sitio web consultando su archivo `robots.txt`. No sobrecargues los servidores con demasiadas peticiones y considera el uso de APIs oficiales cuando estén disponibles. El scraping debe realizarse de manera responsable y dentro de los términos de servicio.

El scraping se menciona aquí porque es **una forma legítima de "Extract"** en el flujo ETL, especialmente útil cuando trabajamos con datos públicos que no están disponibles en formatos estructurados.

Conexión a bases de datos SQL y SAS

Las bases de datos relacionales son el corazón del almacenamiento empresarial de datos. Python ofrece múltiples bibliotecas para conectarse con estos sistemas, permitiendo extraer datos directamente desde entornos de producción.

Trabajando con bases de datos SQL

```
import sqlite3
import pandas as pd

# Establecer conexión con la base de datos
conn = sqlite3.connect("northwind.db")

# Ejecutar consulta SQL y cargar en DataFrame
query = """
SELECT
    OrderID,
    CustomerID,
    OrderDate,
    ShipCountry
FROM Orders
WHERE OrderDate >= '2023-01-01'
"""

df = pd.read_sql(query, conn)

# Cerrar conexión
conn.close()

# Explorar los datos
print(f"Registros extraídos: {len(df)}")
print(df.head())
```

Ventajas de la conexión directa

- Acceso a datos actualizados en tiempo real
- Aprovechamiento del poder de SQL para filtrar y agregar
- Reducción de transferencia de datos innecesarios
- Integración con sistemas empresariales existentes

Conectores para diferentes sistemas

SQLite sqlite3 (incluido en Python) Base de datos ligera ideal para desarrollo y proyectos pequeños	MySQL / MariaDB mysql-connector-python Sistema robusto para aplicaciones web y empresariales
PostgreSQL psycopg2 Base de datos avanzada con características empresariales	SQL Server pyodbc Conector para ecosistema Microsoft

Conexión con sistemas SAS

Para organizaciones que utilizan SAS como plataforma principal de análisis, Python ofrece SASPy, una biblioteca que permite la integración entre ambos entornos:

```
import saspy

# Establecer sesión SAS (requiere configuración previa)
sas = saspy.SASsession(cfgname='default')

# Leer dataset SAS en DataFrame de Pandas
df = sas.sasdata2dataframe(
    table='customers',
    libref='work'
)

# Cerrar sesión
sas.endsas()
```



Nota importante

La conexión con SAS requiere un entorno SAS configurado y licenciado. Es ideal para empresas que almacenan datos históricos en SAS pero quieren aprovechar las capacidades modernas de Python para análisis y visualización.

En el contexto ETL, **conectar directamente con bases de datos permite extraer los datos más recientes** sin necesidad de exportaciones manuales, manteniendo la automatización y consistencia del proceso.

Etapa 2: Transformación (Transform)

La transformación es el corazón del proceso ETL. Aquí es donde los datos crudos se convierten en información útil y confiable. Esta etapa garantiza la calidad, consistencia y estructura adecuada para el análisis posterior o la carga en sistemas de destino.

Operaciones típicas de transformación

1	Manejo de valores faltantes Identificar y tratar valores nulos mediante imputación, eliminación o estrategias específicas del dominio <pre>df.fillna(df.mean()) # Rellenar con media df.dropna() # Eliminar filas con nulos</pre>
2	Unión de datasets Combinar múltiples fuentes de datos mediante joins, concatenaciones o merges <pre>pd.merge(df1, df2, on='id', how='left')</pre>
3	Creación de columnas derivadas Calcular nuevas variables basadas en columnas existentes para enriquecer el análisis <pre>df.assign(total=lambda x: x['precio'] * x['cantidad'])</pre>
4	Agrupación y agregación Resumir datos mediante operaciones de grupo para obtener métricas clave <pre>df.groupby('categoria')['ventas'].sum()</pre>
5	Aplicación de funciones personalizadas Transformaciones complejas que requieren lógica específica del negocio <pre>df['col'].apply(lambda x: funcion_custom(x))</pre>

Ejemplo de pipeline de transformación

Pandas permite encadenar múltiples operaciones de manera elegante y legible, creando pipelines eficientes:

```
# Pipeline completo de transformación
resultado = (
    df
    .query('precio > 0') # Filtrar precios válidos
    .assign(
        # Calcular precio final con descuento
        precio_final=lambda x: x["precio"] * (1 - x["descuento"]),
        # Categorizar por rango de precio
        rango_precio=lambda x: pd.cut(
            x["precio_final"],
            bins=[0, 100, 500, float('inf')],
            labels=['Bajo', 'Medio', 'Alto']
        )
    )
    .groupby(['categoria', 'rango_precio']) # Agrupar por categoría y rango
    .agg({
        'precio_final': ['mean', 'sum', 'count'],
        'cantidad': 'sum'
    })
    .round(2) # Redondear resultados
)
```

Transformación para análisis

Cuando el objetivo es explorar y visualizar datos, la transformación prepara el terreno para el **EDA (Análisis Exploratorio)**.

- Normalización de escalas
- Creación de variables visuales
- Preparación de datos para gráficos

Transformación para automatización

En flujos automatizados, la transformación se enfoca en **consistencia y reproducibilidad**, terminando antes de la fase de análisis.

- Aplicación de reglas de negocio
- Validación de calidad de datos
- Preparación para carga en destino

 **Punto clave de decisión**

Si el propósito es **análisis**, después de esta fase viene el EDA. Si el propósito es **automatización**, aquí termina el flujo creativo antes de cargar/exportar los datos procesados.

Etapa 2.5: Análisis Exploratorio de Datos (EDA)

¿Cuándo aparece el EDA?

👉 **Solo en proyectos orientados al análisis**, no en automatización pura. El EDA es una fase interpretativa que se sitúa entre la transformación de datos y la generación de reportes finales o modelos predictivos.

Propósito del análisis exploratorio



Comprender los datos

Familiarizarse con la estructura, distribuciones y características fundamentales del dataset antes de modelar o generar conclusiones



Detectar anomalías

Identificar errores de calidad, outliers, valores inconsistentes o patrones inesperados que requieren atención



Descubrir relaciones

Encontrar correlaciones, patrones y relaciones entre variables que pueden ser clave para el análisis o la modelización



Validar hipótesis

Crear visualizaciones que confirmen o refuten suposiciones iniciales sobre el comportamiento de los datos

Técnicas y herramientas comunes

Ejemplo práctico con Seaborn

```
import seaborn as sns
import matplotlib.pyplot as plt

# Matriz de correlación
plt.figure(figsize=(10, 8))
sns.heatmap(
    df.corr(),
    annot=True,
    cmap='coolwarm',
    center=0
)
plt.title('Correlaciones entre variables')
plt.show()

# Pairplot para múltiples variables
sns.pairplot(
    df[["age", "fare", "survived"]],
    hue="survived",
    diag_kind="kde"
)
plt.show()
```

Análisis univariado

- Histogramas de distribución
- Box plots para detectar outliers
- Estadísticas descriptivas

Análisis bivariado

- Scatter plots de correlación
- Gráficos de barras agrupadas
- Matrices de correlación

Análisis multivariado

- Pairplots completos
- Reducción de dimensionalidad (PCA)
- Facet grids para segmentación



Diferencia fundamental

El EDA **no transforma** los datos en el sentido técnico del ETL. Su función es **interpretarlos y visualizarlos** para extraer insights que guíen decisiones posteriores. Es una fase exploratoria y creativa, no productiva en términos de automatización.

En resumen, el EDA es el puente entre tener datos limpios y generar conocimiento accionable. Es donde la ciencia de datos se convierte en arte analítico.

Etapa 3: Carga o Salida (Load)

La etapa de carga es el punto final del proceso ETL, donde los datos transformados se exportan a su destino final. Esta fase garantiza que los resultados del análisis o procesamiento estén disponibles en formatos accesibles y útiles para diferentes audiencias.

Formatos de exportación más comunes

Formato	Código de Ejemplo	Uso y Características
CSV (Comma-Separated Values)	<pre>df.to_csv("reporte.csv", index=False)</pre>	Formato universal compatible con cualquier herramienta. Ideal para compartir datos de manera simple y ligera.
Excel (XLSX)	<pre>df.to_excel("reporte.xlsx", index=False, sheet_name="Datos")</pre>	Perfecto para usuarios no técnicos. Permite múltiples hojas y formato condicional.
HTML (Página Web)	<pre>df.to_html("reporte.html", index=False)</pre>	Útil para reportes web o para enviar por correo con formato visual preservado.
JSON (JavaScript Object Notation)	<pre>df.to_json("datos.json", orient="records", indent=2)</pre>	Ideal para APIs, aplicaciones web y sistemas que requieren datos estructurados jerárquicamente.

Exportación de visualizaciones

Guardar gráficos estáticos

```
import matplotlib.pyplot as plt

# Crear visualización
plt.figure(figsize=(10, 6))
df.plot(kind='bar')
plt.title('Ventas por categoría')
plt.xlabel('Categoría')
plt.ylabel('Ventas ($)')

# Guardar en alta resolución
plt.savefig("grafico_ventas.png", dpi=300, bbox_inches='tight', transparent=False)

plt.close()
```

Formatos de imagen

- PNG:** ideal para presentaciones y documentos (con transparencia)
- JPG:** menor tamaño de archivo, sin transparencia
- SVG:** vectorial, perfecto para escalado sin pérdida de calidad
- PDF:** para impresión profesional de alta calidad

Exportar notebooks completos como reportes

Jupyter Notebooks pueden convertirse en reportes profesionales usando `nbconvert`:

```
# Convertir notebook a HTML
jupyter nbconvert --to html Clase12.ipynb --output reporte_final.html

# Convertir a PDF (requiere LaTeX instalado)
jupyter nbconvert --to pdf Clase12.ipynb

# Convertir a Markdown
jupyter nbconvert --to markdown Clase12.ipynb
```

Buenas prácticas de exportación

- Incluir metadatos como fecha de generación y autor
- Usar nombres de archivo descriptivos con timestamp
- Documentar el formato y estructura de los datos exportados
- Validar la integridad de los archivos generados
- Considerar la compresión para datasets grandes

La fase de carga cierra el ciclo ETL, entregando **productos de datos listos para consumo** por stakeholders, sistemas automatizados o análisis posteriores.

Automatización de reportes y scripts

La automatización transforma procesos manuales repetitivos en flujos de trabajo programados que se ejecutan sin intervención humana. Esto garantiza consistencia, ahorra tiempo y reduce errores, permitiendo que los equipos se enfoquen en tareas de mayor valor.

Estructura de un script ETL automatizable

```
import pandas as pd
from datetime import datetime
import os

def extraer_datos():
    """Extrae datos desde la fuente"""
    df = pd.read_csv("datos_ventas.csv")
    return df

def transformar_datos(df):
    """Aplica transformaciones y limpieza"""
    # Filtrar datos válidos
    df_limpio = df[df['monto'] > 0].copy()

    # Agregar fecha de procesamiento
    df_limpio['fecha_proceso'] = datetime.now()

    # Calcular métricas por categoría
    resumen = df_limpio.groupby('categoria').agg({
        'monto': ['sum', 'mean', 'count'],
        'unidades': 'sum'
    }).round(2)

    return resumen

def cargar_datos(df, nombre_archivo):
    """Exporta el resultado"""
    # Crear carpeta de reportes si no existe
    os.makedirs("reportes", exist_ok=True)

    # Nombre con timestamp
    timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
    ruta = f"reportes/{nombre_archivo}_{timestamp}.xlsx"

    # Guardar con formato
    df.to_excel(ruta, sheet_name="Resumen")
    print(f"
```

Cierre: ETL + EDA como caminos complementarios



Hemos recorrido el flujo completo de trabajo con datos, desde la extracción hasta la automatización. Es momento de consolidar estos conceptos y entender cómo se complementan en el ecosistema de ciencia de datos.

Resumen general del proceso

01	02
ETL como proceso base El ETL es el fundamento de todo proyecto de datos , proporcionando la estructura sistemática para obtener, procesar y almacenar información de manera confiable.	Garantía de calidad en Transform La calidad y consistencia de los datos se aseguran principalmente en la etapa de transformación , donde aplicamos reglas de negocio y limpieza.
03	04
EDA para análisis interpretativo El análisis exploratorio aparece cuando el objetivo es comprender, explorar y visualizar patrones en los datos, no en todos los proyectos.	Automatización para reproducibilidad La automatización se implementa cuando necesitamos repetir procesos y generar salidas confiables sin intervención manual continua.

Dos tipos fundamentales de proyectos de datos

Proyectos de análisis (EDA)

Objetivo principal: Comprender, explorar y comunicar insights

- Exploración visual de patrones
- Estadística descriptiva e inferencial
- Generación de hipótesis
- Reportes analíticos con visualizaciones
- Presentaciones para stakeholders

Énfasis: Visualizaciones, interpretación y storytelling con datos

Proyectos de automatización

Objetivo principal: Estandarizar y producir salidas confiables

- Pipelines reproducibles
- Reportes programados periódicamente
- Integración con sistemas empresariales
- Validación automática de calidad
- Monitoreo y alertas

Énfasis: Transformación robusta y carga confiable sin intervención manual

Lecciones clave para aplicar

Define el objetivo desde el inicio Antes de comenzar, clarifica si tu proyecto busca análisis exploratorio o automatización, ya que esto determinará las herramientas y enfoques a utilizar.	La calidad de datos es no negociable Invierte tiempo en la fase de transformación. Datos de mala calidad producen análisis erróneos o automatizaciones inestables.
Documenta tu proceso Tanto en análisis como en automatización, la documentación clara del código y las decisiones tomadas es fundamental para la reproducibilidad.	Itera y mejora continuamente Los procesos ETL y EDA no son estáticos. Refina tus pipelines y análisis basándote en feedback y nuevos requerimientos.

Conclusión: Dominar ETL y EDA te permite **construir una base sólida en ciencia de datos**, capacitándote para extraer valor de cualquier fuente de información y comunicar insights de manera efectiva. Ya sea automatizando procesos o descubriendo patrones ocultos, estas habilidades son fundamentales en el arsenal de cualquier profesional de datos.